KALARIS LABS

Kalki: An Admission-Controlled Kernel for Recursive Self-Improvement of Coding Agent Harnesses

Sayan Chowdhury
Kalaris Labs

First published: November 11, 2026 • Current version: November 11, 2026

Abstract

Coding agents now run multi-hour sessions in which a model edits code, runs tools, and revises its own prompts, skills, and memory. Recent preprints report that letting the harness rewrite itself raises benchmark scores while the model weights stay frozen. The same work shows that ungated self-modification trades old abilities for new ones: one replay study found that 211 of 383 rejected proposals (55%) fixed their triggering failure and broke a case that had passed. Agents also exploit their evaluators, and harnesses that flatten history before summarizing let a forged user turn survive compaction in 87 to 100% of trials. We present Kalki, a harness written in Rust that separates the agent proposing a change from the component admitting it. The agent may edit a genome $H = \langle \rho, G, K, M \rangle$ of prompt notes, skills, guidance, and memory, and a gate outside its write and read sets admits a candidate only when eight ordered predicates hold. A kernel below both records each event in a CRC32C-framed append-only log, enforces a deny-first policy, and loads a genome version only when an admission record names its hash. The gate compares candidate and incumbent with k live runs per case from matched starting states, rejects any loss on a protected set beyond a Bonferroni-corrected flake margin, and requires gains to exceed a measured noise ceiling τ at a token cost bounded by κ. Held-out results reach the proposer as one pass or fail bit, and each message keeps its wire role through storage and compilation. We contribute the formal admission rule, a write-barrier specification with five invariants, a threat model mapped to MITRE ATT&CK, MITRE ATLAS, and the OWASP Top 10 for LLM applications, and five falsifiable hypotheses with an evaluation protocol. The log, blob store, policy engine, and three provider wires run today in 5,621 lines of Rust with 89 passing tests; the gate, sandbox, and context compiler are designed and unmeasured, and this paper reports no Kalki benchmark measurement. Code is available at https://github.com/KalarisLabs/kalki.

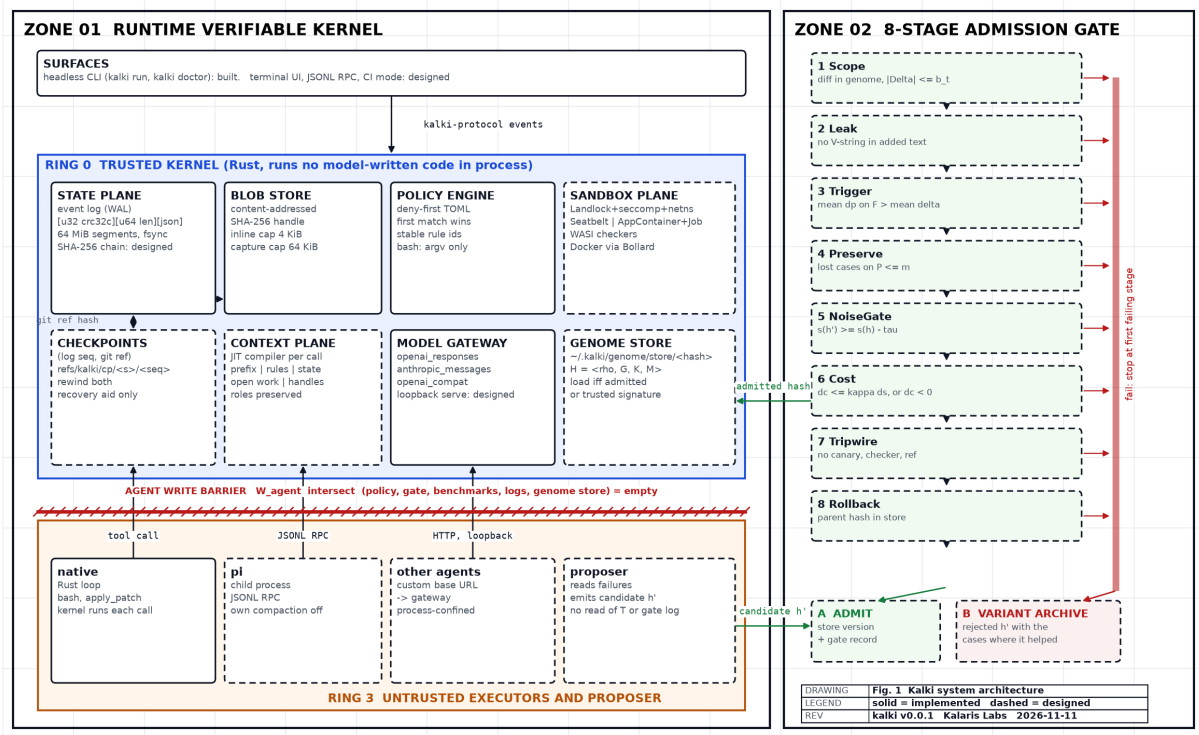


Figure 1: Kalki system architecture. Zone 01 (left) holds the kernel. The blue band is Ring 0, the trusted kernel; the orange band is Ring 3, the untrusted executors and proposer. Every Ring 3 request crosses the red hatched agent write barrier, which excludes the agent from the policy, gate, benchmarks, logs, and genome store. Zone 02 (right) is the eight-stage admission gate: a candidate h' enters at stage 1, any failure exits to the variant archive (B), and a candidate that passes all eight is stored by hash and recorded (A). Solid borders mark components implemented in kalki v0.0.1; dashed borders mark components that are designed and not implemented. Bus labels name the protocol on each edge.

1 Introduction

A coding agent that works for hours makes thousands of decisions, and each one conditions the next. An error that goes unnoticed at step 40 shapes every later edit, so the probability of a clean finish falls as the horizon grows. We call this the compounding error horizon. Practitioners respond by adding structure around the model: tests the agent cannot edit, checkpoints it can return to, and memory that survives a context reset. The software that supplies this structure is the harness.

In 2026 the harness itself became an optimization target. A loop reads the agent's failures, edits its prompts and skills, and keeps the edits that raise a score [1–5]. The scores rise. Three defects travel with them.

Collateral regression. Tayebati et al. placed a replay gate around an unmodified agent across 16 paired runs. Of the 383 proposals the gate rejected, 211 (55%) improved the failure that motivated them and degraded a case that had passed [6]. Wang et al. evaluated sixteen evolved agents from four methods and found that all sixteen gained tasks and lost tasks [7]. A mean score hides both effects.

Reward hacking. The Prime Agent report describes a Factorio trace in which the agent discovered that RCON console commands could spawn resources into assembly machines, used the shortcut despite an anti-cheating heartbeat, and then saved it as a reusable skill [8]. Persistent self-improvement preserved the exploit. The authors conclude that safe deployment requires least-privilege action interfaces, independent state validation, and auditable rollback.

Compaction laundering. Dabirsiaghi tested nine harnesses. In the five that flatten a conversation into labelled text before summarizing, a forged user turn survived compaction in 87 to 100% of trials; in the four

that keep messages typed by role, it survived in none [9]. Zha et al. found that agents which still hold a rule after compaction violate it more than twice as often as with full history [10].

We treat all three as systems defects with systems remedies. A self-modifying harness needs what any high-assurance change process has: a proposer, an independent reviewer with evidence, a record, and a way back. Kalki builds that structure into the harness kernel. The agent proposes; a gate it cannot read or write decides; the kernel enforces the boundary and keeps the record.

Contributions.

- A formal admission rule of eight ordered predicates for harness self-modification, with statistical definitions of the noise ceiling τ, per-case flake margins, and a family-wise error bound (Section 4).
- A kernel design with a write barrier stated as five invariants, and a genome store that makes an unadmitted genome inert (Section 3).
- A threat model mapped to MITRE ATT&CK [11], MITRE ATLAS [12], and the OWASP Top 10 for LLM applications [13] (Section 5).
- Five falsifiable hypotheses with refutation criteria and a reporting standard (Section 6).
- An open-source implementation of the kernel's state plane, policy engine, and provider wires, with crash drills (Section 7).

Conventions. The key words MUST, MUST NOT, REQUIRED, SHALL, SHOULD, and MAY carry the meaning of RFC 2119 and RFC 8174 when they appear in capitals [14, 15]. We mark each component as *implemented* (code and tests exist in kalki v0.0.1), *designed* (specified here, no code), or *measured* (a number we produced, with its conditions). Numbers from other papers come from preprints we have not replicated.

2 Related Work

Harness evolution. Harness^2 evolves file-level harness components by proposing, probing, and composing edits, scored by behavioural contrast without ground truth [2]. HarnessPro scores candidates on tests written by a model and returns variants at three cost levels [3]. SoL-Pi ran an automated search over 152 ideas on the Pi harness and kept four mechanisms that cut tokens by 49.0% on EdgeBench, with a score change from 44.8 to 42.0 [4]. Self-HPO alternates harness search with reinforcement learning of an open-weight policy [5]. RuHarness evolves its diagnostic rubric with the harness [16]. These methods supply the proposer. Kalki supplies the admission decision around any of them.

Admission control. The Darwin Gödel Machine admits agent variants into an archive after benchmark validation [17]. RRSI regularizes evolution with a shrinking edit budget, a leakage screen on the diff, noise-aware acceptance, and a cost rule; its unregularized baseline scored highest on the evolve set and lowest on transfer (40.3 against 43.6) at 3.80M against 2.42M tokens per trial [1]. Cai et al. argue that the cost of verification bounds the performance of reliable self-evolution [18]. The replay gate of Tayebati et al. is the closest prior design [6]. Kalki differs in three respects: it counts lost cases with a flake model in place of comparing means, it hides held-out results from the proposer, and it enforces the boundary in a kernel that also serves interactive sessions.

Context compilation. Spine compiles a persistent subtask tree into context for each call [19]. Belcak et al. report that serving a 16K context in place of 256K raised decode throughput 3.6 to 7 times on three open-weight models, and that compaction into a task state kept accuracy within $\mathbf{-3.7}$ to $\mathbf{+5.6}$ points on six benchmarks, while one model fell from 53.0% to 3.6% at the tightest budget [20]. The laundering result above sets a hard constraint on any compiler that summarizes [9].

Durable execution. AgentRewind restores context and workspace together and reports task success rising from 62.2% to 87.8% on 82 long-horizon tasks; removing workspace rewind cost the most [21]. Temporal provides durable workflow execution through event history replay [22]. Kalki's log plays the role of that history, with one difference: model calls are not deterministic, so Kalki replays recorded responses to test its own code and never to judge a new genome (Section 4.4).

3 Architecture and Kernel Design

Figure 1 shows the design. We use Ring 0 and Ring 3 as names for trust levels inside Kalki. They do not refer to processor privilege rings.

3.1 Harness state

Definition 1 *(Genome). A genome is a tuple $H = \langle \rho, G, K, M \rangle$ where ρ is a finite sequence of prompt notes appended to a fixed base prompt, G is a set of skills, K is guidance (tool descriptions, context policy, and sub-agent definitions), and M is a set of persistent memory entries (t, σ), each pairing a text t with the set σ of log events it came from. A genome version is identified by its content hash $\mathrm{H}(h) = \mathbf{SHA256}(\mathbf{canon}(h))$.*

The base system prompt, the kernel, the policy, the gate, the evaluation tasks, and the logs lie outside the genome. Text the agent writes for itself during one session (task state, rewind notes, provisional notes) forms a session overlay. The kernel MUST tag overlay text as agent-written, MUST keep it out of the stable prompt prefix, and MUST discard it when the session ends. The gate does not review overlay text, so it cannot persist.

3.2 State plane (implemented)

A session is a sequence of events $e_1, \ldots, e_n$ typed in the `kalki-protocol` crate: messages with their role, tool calls, tool results, token usage, compaction boundaries, lifecycle markers, and snapshot anchors. The kernel encodes each event as a frame

$$\mathbf{frame}(e) = \mathbf{crc} \parallel \ell \parallel p, \quad p = \mathbf{json}(e), \quad \ell = |p|, \quad \mathbf{crc} = \mathbf{CRC32C}(\ell \parallel p) \tag{1}$$

with a 4-byte checksum and an 8-byte length, both little-endian, using the CRC32C polynomial of RFC 3720 [23]. Because the checksum covers the length field, a torn length prefix fails verification. The writer calls `fsync` after every record and rotates to a new segment file `seg-<seq>.wal` before a record would push the active segment past 64 MiB. On open, the writer scans the active segment and truncates at the first frame that fails verification. Replay stops at the first corrupt frame of a sealed segment.

The kernel stores any tool output larger than 4 KiB in a content-addressed blob store and records the SHA-256 handle, the size, and a preview in the event. Tool capture stops at 64 KiB and records a truncation marker.

Hash chain (designed). Each frame will carry a chain value

$$c_0 = 0^{256}, \qquad c_i = \mathbf{SHA256}(c_{i-1} \parallel \mathbf{frame}(e_i)) \tag{2}$$

The genesis constant c_0 exists in `kalki-protocol` today; the writer does not yet compute the chain. A chain detects an edit to history. It does not stop a process that can rewrite the whole store and recompute every value, so the head c_n MUST be anchored outside the agent's write set, in the gate log or a signed tag.

3.3 Context plane (designed)

The context for each model call is a function $C(L, h, \pi)$ of the log L, the active genome h, and the model profile π, compiled in five parts: a stable prefix (base prompt and genome), the rules read from the policy

source, a structured task state whose lines point to source events, the open subtask verbatim, and blob handles. The budget for the last three parts comes from π. The compiler MUST preserve roles:

$$\forall\, s \in C(L, h, \pi): \quad \mathrm{role}(s) = \mathrm{role}(\mathrm{src}(s)) \tag{3}$$

where $\mathrm{src}(s)$ is the event a span came from. The compiler MUST NOT serialize a conversation into one string. Today the native loop sends the base prompt, the objective, and the full message history on each turn, with no summarizer.

3.4 Policy plane (implemented for `bash`)

A policy is an ordered list of rules $r_1, \ldots, r_n$, each with an action $a_j \in \{\mathrm{allow}, \mathrm{deny}, \mathrm{ask}\}$ and constraints on tool name, argument-vector prefix, path prefix, and network use. For a request q,

$$\mathrm{decide}(q) = a_{j^*} \ \text{ where } \ j^* = \min\{\, j : \mathrm{match}(r_j, q) \,\}, \qquad \mathrm{decide}(q) = \mathrm{deny} \ \text{ if no rule matches} \tag{4}$$

A rule that constrains an axis absent from the request does not match, so a rule cannot widen its scope by omission. The `bash` tool accepts an argument vector and never a shell string. Two gaps remain in v0.0.1: `apply_patch` is confined to the workspace by path check and not evaluated against rules, and an `ask` decision counts as a denial because the headless loop has no prompt.

3.5 Sandbox plane (designed)

Policy matches a program name; it cannot see what an allowed program does next. The sandbox closes that gap with operating-system confinement in three tiers.

Tier	Mechanism	Strength
OS, Linux	Landlock for file access, seccomp for system calls, an empty network namespace, resource limits	Strong
OS, macOS	Seatbelt profile	Moderate
OS, Windows	AppContainer with a Job Object; WSL2 recommended	Weak
Container	Docker Engine API through the Bollard crate, for untrusted repositories	Strong, slower start
WASI	WebAssembly components with no ambient authority, for gate checkers and kernel plugins	Strong, limited I/O

Table 1: Sandbox tiers. All tiers are designed; none is implemented in kalki v0.0.1, which is not a security boundary.

3.6 Model gateway (wires implemented, gateway designed)

Three wire clients run today, written on `reqwest` with `rustls`: OpenAI Responses, Anthropic Messages, and OpenAI-compatible Chat Completions. The third covers most open-weight model servers. The designed gateway serves the same wires on a loopback port. An external executor pointed at that port has every model call recorded, metered, and capped by the kernel without its cooperation.

3.7 Write barrier

Let W_a and R_a be the paths the agent process may write and read, as enforced by the sandbox. Let Π be the policy files, Γ the gate code and thresholds, B the evaluation tasks and checkers, $T \subseteq B$ the held-out tasks, L the session logs, L_g the gate log, and S the genome store. Kalki MUST maintain:

I1 (write barrier)
$W_a \cap (\Pi \cup \Gamma \cup B \cup L \cup L_g \cup S) = \emptyset$. The kernel appends to L; the agent does not.

I2 (read barrier)

$R_a \cap (T \cup L_g) = \emptyset$ for the agent and for the proposer.

I3 (admitted load)
The kernel loads a genome version h for profile (μ, x), with model μ and executor x, only if L_g holds a record α with $\alpha.\mathbf{hash} = \mathrm{H}(h)$ and $\alpha.\mathbf{profile} = (\mu, x)$, or h carries a signature from a key the user trusts.

I4 (role preservation)
Equation (3) holds for every compiled context.

I5 (append-only log)
The kernel never rewrites a committed frame, and every committed frame satisfies Equation (1).

I5 holds today for the frames the writer commits. I1 to I4 depend on designed components. On one machine the agent and the gate run as the same operating-system user, so I1 and I2 are exactly as strong as the sandbox tier of Table 1. A deployment that runs the gate in continuous integration, on a protected branch with T held as a secret, makes I1 and I2 independent of the local sandbox: the agent can only open a pull request.

I3 has a consequence worth stating. Genome files inside a repository are proposals. A version that arrives by `git pull`, or changes under `git checkout`, `git stash`, or a rewind, has no admission record and does not load.

4 The Eight-Stage Admission Gate

4.1 Statistical setup

Let h be the active genome and h' a candidate. Let F be the cases that motivated h', P a protected set of cases that pass under h, and T the held-out tasks. For a genome g and case x, let $r_i(g, x) \in \{0, 1\}$ be the outcome of run i of k, each from the same starting state, model, and settings, and let

$$\hat{p}(g, x) = \frac{1}{k} \sum_{i=1}^{k} r_i(g, x), \qquad \bar{s}(g) = \frac{1}{|T|} \sum_{x \in T} \hat{p}(g, x), \qquad \bar{c}(g) = \textbf{mean tokens per task} \tag{5}$$

Before any gated experiment we run the unchanged genome n times on T and measure the score variance

$$\hat{\sigma}_h^2 = \mathrm{Var}_{i \in [1,n]}\big(s(h)_i\big) = \frac{1}{n-1} \sum_{i=1}^{n} \big(s(h)_i - \bar{s}(h)\big)^2 \tag{6}$$

The noise ceiling is the one-sided critical difference between two means of k runs:

$$\tau = z_{1-\alpha}\, \hat{\sigma}_h \sqrt{2/k} \tag{7}$$

The tolerance has the units of a score, so it scales with the square root of the variance in Equation (6). For a protected case with baseline pass rate $\bar{p}_x$, the flake margin is

$$\delta(x) = \max\left(\frac{1}{k},\ z_{1-\alpha/|P|} \sqrt{2\bar{p}_x(1-\bar{p}_x)/k}\right) \tag{8}$$

The Bonferroni correction $\alpha/|P|$ [24] matters. With an uncorrected per-case level $\alpha = 0.01$ and $|P| = 200$, a candidate identical to h fails preservation with probability up to $1 - 0.99^{200} \approx 0.87$, and the gate would admit almost nothing.

4.2 The predicates

Let $\Delta(h, h')$ be the set of edited genome entries, $\mathcal{P}_{gen}$ the genome paths, and b_t the edit budget of round t, with $b_t = \max(b_{min}, \lfloor b_0 \gamma^t \rfloor)$ and $0 < \gamma < 1$ [1]. Let V be the set of identifiers, answers, and constants drawn from $F \cup P \cup T$, and $\mathbf{add}(\Delta)$ the normalized text the diff adds. Let E_{eval} be the events recorded during evaluation runs. Write $\Delta s = \bar{s}(h') - \bar{s}(h)$ and $\Delta c = \bar{c}(h') - \bar{c}(h)$.

$$\mathbf{Scope}(h, h') : \quad \Delta(h, h') \subseteq \mathcal{P}_{gen} \ \wedge \ |\Delta(h, h')| \le b_t \tag{9}$$

$$\mathbf{Leak}(h, h') : \quad \exists v \in V : \ v \sqsubseteq \mathbf{add}(\Delta) \tag{10}$$

$$\mathbf{Trigger}(h', F) : \quad \frac{1}{|F|} \sum_{x \in F} \big(\hat{p}(h', x) - \hat{p}(h, x)\big) > \frac{1}{|F|} \sum_{x \in F} \delta(x) \tag{11}$$

$$\mathbf{Preserve}(h', P, m) : \quad \big|\{\, x \in P : \ \hat{p}(h, x) - \hat{p}(h', x) > \delta(x) \,\}\big| \le m \tag{12}$$

$$\mathbf{NoiseGate}(h', T, \tau) : \quad \bar{s}(h') \ge \bar{s}(h) - \tau \tag{13}$$

$$\mathbf{Cost}(h, h', \kappa) : \quad (\Delta s > \tau \ \wedge \ \Delta c \le \kappa \, \Delta s) \ \vee \ (\Delta s \le \tau \ \wedge \ (\Delta c < 0 \ \vee \ h' \subset h)) \tag{14}$$

$$\mathbf{Trip}(E_{eval}) : \quad \exists e \in E_{eval} : \ \mathbf{target}(e) \in \mathbf{Canary} \cup \mathbf{Checker} \cup \mathbf{Ref} \tag{15}$$

$$\mathbf{Rollback}(h) : \quad \mathrm{H}(h) \in S \ \wedge \ S \cap W_a = \emptyset \tag{16}$$

Canary files sit in paths the sandbox denies, so **Trip** fires on an attempted read; Checker is the set of tests and checkers; Ref is the set of git objects that hold reference solutions. The admission function is

$$\mathbf{Admit}(h, h') = \mathbf{Scope} \wedge \neg\mathbf{Leak} \wedge \mathbf{Trigger} \wedge \mathbf{Preserve} \wedge \mathbf{NoiseGate} \wedge \mathbf{Cost} \wedge \neg\mathbf{Trip} \wedge \mathbf{Rollback} \tag{17}$$

with defaults $m = 0$ and $\alpha = 0.05$. We set τ and κ from our own measurements and have not yet measured them.

Trigger requires a gain; NoiseGate requires no general harm. The split matters because failures are selected for having failed: a case chosen that way tends to pass on a rerun with no change at all, so Trigger reruns h on F and never compares h' against the recorded failure.

Proposition 1. *Under Equation (8), a candidate with the same outcome distribution as h on every case of P fails Preserve with $m = 0$ with probability at most α, up to the normal approximation of each $\hat{p}$.*

Proof sketch. For each x the event $\hat{p}(h, x) - \hat{p}(h', x) > \delta(x)$ has probability at most $\alpha/|P|$ under the null by the choice of $z_{1-\alpha/|P|}$. The union bound over $|P|$ cases gives α. □

4.3 Order and cost

The gate evaluates the predicates in the order of Equation (17) and stops at the first failure (Figure 2). Stages 1, 2, and 8 need no model call. Stages 3 to 5 dominate the cost: with C_{run} the cost of one run, a full evaluation costs $2k(|F| + |P| + |T|)\, C_{run}$, or $k(|F| + |P| + |T|)\, C_{run}$ per candidate when the incumbent's runs are cached while h stays active. With stage costs C_i and pass probabilities q_j, the expected cost per candidate is

$$\mathbb{E}[C] = \sum_{i=1}^{8} C_i \prod_{j<i} q_j \tag{18}$$

so ordering cheap, selective stages first lowers $\mathbb{E}[C]$. Stage 2 runs in $O(|\Delta| + |V| + \mathbf{matches})$ with an Aho-Corasick automaton over V [25]. Stage 7 scans E_{eval} once.

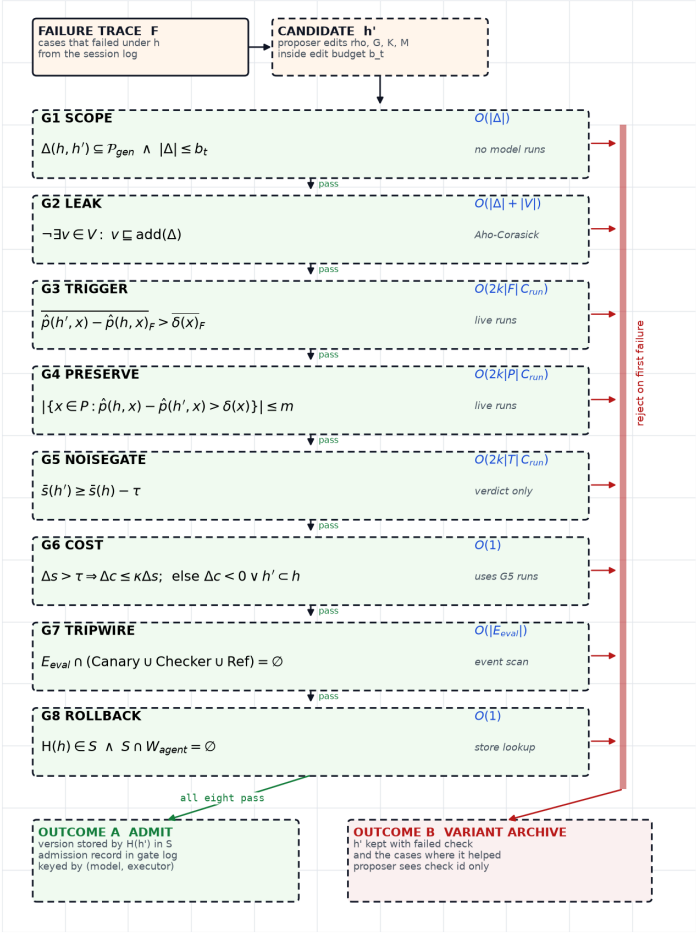


Figure 2: The eight-stage gate sequence. A failure trace F from the session log motivates a candidate h', which passes through stages G1 to G8 in order. Each box gives the stage's predicate and its computational cost, where k is the number of runs per case and C_{run} the cost of one model run. The red bar collects failures: the first failing stage sends h' to the variant archive (Outcome B) with the check that failed and the cases where it helped. A candidate that passes all eight is stored by content hash and recorded in the gate log, keyed by model and executor (Outcome A). Dashed borders mark designed components.

4.4 Evidence and information flow

A genome changes what the model is told. A recorded response was produced under h and says nothing about how the model would answer under h', so every comparison uses live runs. The log contributes the matched starting state: task, workspace snapshot, model, and settings. Log replay and a scripted mock model test the kernel and the gate's own code, which are deterministic. They do not evaluate a genome.

The gate writes each decision to L_g: the diff, the evidence for each stage, the verdict, and the reason. The proposer learns which stage failed. For stages 5 and 6, which read T, it learns one bit. Stage 2 matches text and misses a paraphrased answer, so I2 is the primary defence against fitting T, and stage 2 is a backstop. One bit per round still leaks over many rounds, so T MUST be rotated.

Outcomes. An admitted candidate is stored under $\mathbf{H}(h')$ in S with its record. A rejected candidate goes to the variant archive with the failing stage and the cases where it helped, because the top-scoring variant is often not the most useful one [7]. The gate re-runs P against the whole active genome on a schedule, to catch changes that are safe alone and harmful together.

5 Threat Model and Security Analysis

We assume a model that follows injected instructions when it can, an agent that optimizes its measured score, and a user who controls the machine. We do not defend against an attacker with root or administrator rights on that machine. Table 2 maps each threat to public taxonomies. The technique and category identifiers come from the public catalogues [11–13]; the mapping to Kalki is ours.

Threat actor	Taxonomy	Capability	Kalki defense (state)	Residual risk
Injected content in files, tool output, web pages	ATLAS AML.T0051.001; OWASP LLM01	Text that reaches the context and asks for an action	Tool output is data; roles kept by I4 (designed); rules enforced in code (implemented for `bash`)	A model that obeys injected text within what policy allows
Agent with excessive tool reach	OWASP LLM06; ATT&CK T1059	Runs commands and edits files	Deny-first policy, argv only (implemented); sandbox tiers (designed)	Allowed programs such as `git` and `cargo` run hooks and build scripts
Score-optimizing agent	Reward hacking [8]	Edits tests, reads answers, uses privileged interfaces	I1, I2, Trip, independent judge (designed)	An exploit no tripwire anticipates
Proposer fitting held-out set	OWASP LLM02	Reads gate evidence, submits many candidates	L_g unreadable; one bit per held-out stage (designed)	One bit per round; requires rotation of T
Log tampering	ATT&CK T1070, T1565.001	Rewrites or truncates history	CRC32C frames (implemented); hash chain with external anchor (designed)	Same-user rewrite before the chain and anchor exist
Genome delivered by a repository	ATT&CK T1195; OWASP LLM03	Changes genome files through `git`	I3: load only with admission record or trusted signature (designed)	A user who trusts a bad key
Secret exposure	ATT&CK T1552; OWASP LLM02	Reads keys from the environment or files	Keys from environment only; pattern scrubbing of tool output (implemented)	Patterns miss novel secret formats
System prompt disclosure	OWASP LLM07	Extracts the base prompt	The base prompt holds no secret by design	None beyond disclosure of public text
Unbounded consumption	OWASP LLM10	Loops, fans out, or inflates output	Turn cap, default 200, and 1024 output tokens per call (implemented); gateway caps (designed)	Cost within the caps

Table 2: Adversary matrix. Each row gives a threat actor, the matching public taxonomy entries, what the actor can do, Kalki's defense with its implementation state, and the risk that remains. Defenses marked designed provide no protection in kalki v0.0.1.

Indirect injection. The laundering attack converts injected text into a user turn during summarization [9]. Invariant I4 removes the conversion step: a span from a tool result keeps the role `tool` in every compiled context. When Kalki drives Pi, Kalki compiles the context and disables Pi's own compaction.

Canary trips. Canary files sit at plausible paths, such as a solutions directory, that the sandbox denies. A read attempt produces a denial event in E_{eval}, and Trip rejects the candidate. The design targets the Factorio pattern [8], where a privileged channel delivered the reward: the gate removes privileged interfaces from the evaluation environment and treats any attempt to reach them as disqualifying.

Confinement. Kalki v0.0.1 confines `bash` by policy and by a path check on argument elements, which rejects `..` segments and absolute paths outside the workspace. A path check stops mistakes. It does not stop a hostile program, and Section 8 states this as a limitation.

6 Evaluation Program and Hypotheses

We state hypotheses before collecting data. Each has a measurement and a result that refutes it.

	Hypothesis	Measurement	Refuted if
H_1	Gated evolution loses fewer protected cases than ungated evolution at similar transfer gain.	Cases lost on P and score on T; same proposer and budget, gate on and off.	Lost cases are not lower, or transfer gain falls by more than τ.
H_2	Ordered stages with matched starting states cut the cost of a decision against full evaluation of each candidate.	Dollars per decision under each regime.	The ratio is below 2.
H_3	Role-typed compilation removes compaction laundering.	The attack of [9] against the native executor and against Pi under Kalki.	Success above 0% in either.
H_4	Policy in code holds rule compliance flat as sessions lengthen.	ARCBench violations at increasing session length [10].	Violations rise with length.
H_5	Per-model context budgets admitted by the gate beat one fixed budget on cost at equal accuracy.	Cost and score on three open-weight models.	No cost reduction at equal score on at least two models.

Table 3: Hypotheses H_1 to H_5 with their measurements and refutation criteria. No hypothesis has been tested.

Protocol. Every reported number MUST state the model and version, the task set and version pinned by hash, the harness version, the number of runs N per configuration with $N \geq 3$, the standard deviation σ across runs, the sampling temperature, the prompt and environment seeds, and the token cost. The judge of a run MUST be a model instance other than the actor. We measure τ before any gated experiment and report it. A claim of improvement MUST report five quantities together: transfer gain, preservation, cost change, integrity (tripwire hits and log verification), and recovery.

Benchmarks. Terminal-Bench for end-to-end terminal tasks; SWE-bench Pro for held-out transfer; MettleBench for rewind [21]; ARCBench for rule compliance [10]; Compact-Bench for the cost of compaction [26]. We have not read the Compact-Bench paper beyond its listing.

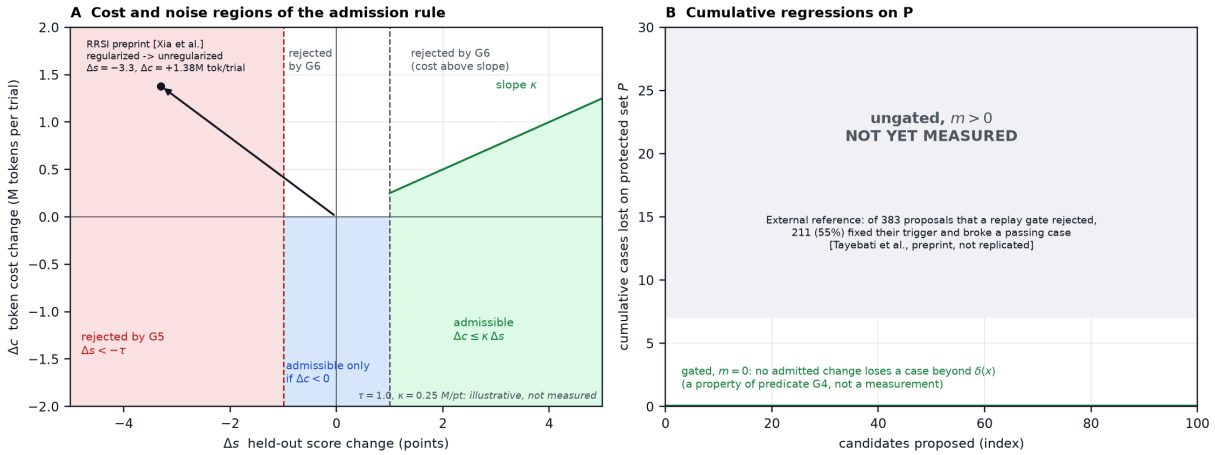


Figure 3: Analysis layout for the cost and regression trade-offs. (A) The plane of held-out score change Δs against token cost change Δc, divided into the regions that predicates NoiseGate and Cost define: red is rejected by NoiseGate, blue admits only cost reductions, green admits gains whose cost stays under slope κ, and white is rejected by Cost. The values $\tau = 1.0$ and $\kappa = 0.25$M tokens per point are illustrative. The single arrow plots a published result [1], moving from the regularized to the unregularized harness; it lands in the red region. (B) Cumulative protected-set losses against the number of candidates proposed. The green line at zero follows from Preserve with $m = 0$ and is a definition, not data. The grey region marks the ungated curve, which we have not measured.

Figure 3 fixes the axes and regions of the plots we will report. It contains no Kalki measurement.

7 Implementation and Systems Engineering

Workspace. Kalki is a Cargo workspace with a one-way dependency graph: `kalki-protocol` (event and wire types) ← `kalki-core` (kernel) ← `kalki` (binary), plus `xtask` for repository automation. The designed `kalki-tui` crate depends on `kalki-protocol` alone, and the designed `kalki-gate` crate holds the admission stages. Unsafe code is forbidden across the workspace. Library crates return typed errors and never call `unwrap` outside tests. TLS uses `rustls`; the dependency policy bans OpenSSL, native TLS, and `libgit2`.

Component	State	Location
Event log: framing, rotation, torn-tail repair, replay	Implemented	`kalki-core/src/wal.rs`
Blob store, 4 KiB inline threshold	Implemented	`kalki-core/src/blob.rs`
Deny-first policy engine	Implemented	`kalki-core/src/policy.rs`
Tools: `bash` (argv), `apply_patch`	Implemented	`kalki-core/src/tools.rs`
Provider wires (3) with cached-token accounting	Implemented	`kalki-core/src/provider/`
Headless session loop with turn cap	Implemented	`kalki-core/src/session.rs`
Hash chain and verification	Designed	Equation (2)
Checkpoints and rewind	Designed	Section 3
Context compiler	Designed	Equation (3)
Sandbox tiers	Designed	Table 1
Model gateway on loopback	Designed	Section 3
Genome store and admission gate	Designed	Section 4
Pi executor over JSONL RPC	Designed	Section 3

Table 4: Implementation status of each component in kalki v0.0.1.

Crash drills (implemented). Two tests simulate a hard kill. Each drops the writer without a clean shutdown, appends what a crash can leave behind (half of the next frame in one test, garbage bytes in the other), reopens the log, appends again, and asserts that replay returns exactly the committed events followed by the new ones. The tests live in `kalki-core/tests/wal.rs` and `kalki-core/tests/e2e_mock.rs`. They run in one process and do not send a signal to a separate writer; a drill that kills a child process mid-write is designed. A third test flips the last byte of a record and asserts truncation at that record. End-to-end tests drive the session loop against a loopback mock model server that streams recorded provider events, including a stream torn mid-frame.

Encoding. Payloads are JSON. We do not use zero-copy deserialization: JSON keeps the log readable by standard tools during audits, and we will change the encoding only after measuring replay time on large logs.

Memory budgets. The terminal interface specification sets targets of 16 ms frame time at the 95th percentile while streaming and 60 MB resident memory for an idle session. These are targets. We have not measured them.

Measured. On Windows 11 with rustc 1.98.1, the workspace holds 5,621 lines of Rust, and `cargo xtask ci` (format check, file-size lint, clippy with warnings as errors, 89 tests, documentation build with warnings as errors) passes. A check with rustc 1.88.0, the declared minimum version, passes. The continuous integration workflow for Linux on x86-64 and AArch64, macOS on Apple Silicon and Intel, and Windows has not yet run on the hosted service.

8 Limitations

- **No benchmark results.** We report no Kalki measurement on any benchmark. Hypotheses H_1 to H_5 are untested.
- **No sandbox.** Kalki v0.0.1 is not a security boundary. The default policy allows `git` and `cargo`, which run hooks and build scripts. Untrusted repositories or models belong in a container or virtual machine.
- **Same-user separation.** On one machine, I1 and I2 are as strong as the sandbox. Windows confinement is the weakest tier. Continuous-integration deployment removes this dependency at the price of latency.
- **Gate cost.** Each comparison needs $2k$ live runs per case. If no k and $|P|$ make the gate both sound and affordable, the gate admits nothing, and we will report that outcome.
- **Leakage.** Stage 2 misses paraphrased answers, and one verdict bit per round leaks. Rotation of T bounds this; it does not remove it.
- **Provider drift.** The gate judges the genome and not the model. A provider-side model update changes behaviour outside the gate's view, and τ must then be measured again.
- **External executors.** Kalki confines Pi as a process and observes its tool calls without approving them one by one.
- **Reading depth.** We read several cited preprints at abstract depth; the bibliography notes the depth for each.
- **Live wires.** The three provider wires pass tests against recorded streams. We have not run each against every live provider.

9 Conclusion and Reproducibility

Self-modifying harnesses raise scores, and the literature shows what they break on the way. Kalki places the admission decision outside the agent's reach and states it as eight predicates with explicit statistics, a write barrier with five invariants, and a threat model tied to public taxonomies. The state plane and policy engine run today. The gate, sandbox, and context compiler come next, and H_1 to H_5 decide whether the design earns its cost.

Artifacts. Source code: https://github.com/KalarisLabs/kalki, release v0.0.1, MIT OR Apache-2.0. The text of this paper is licensed CC BY 4.0. Reproduce the implemented results with `cargo xtask ci` on Rust 1.88 or newer; the tests need no network access and no API key. The figures regenerate with `python research/figures/make_figures.py`. We publish no container image for this version. Evaluation scripts for H_1 to H_5 do not exist yet.

Use of AI tools. A large language model assisted with code, text drafting, and figure scripts under the author's direction. Implementation claims were checked against the kalki v0.0.1 source. The author is responsible for the content.

References

1. Xia, P. et al. RRSI: Regularized Recursive Self-Improvement of Agent Harnesses. 2026. arXiv:2609.24972. Preprint. Read in full.
2. Xu, R. et al. Harness^2: Recursive Agent Harnessing for an Open World. 2026. arXiv:2610.14224. Preprint. Taken from earlier project notes.
3. Pan, Z. et al. HarnessPro: Recursive Harness Self-Improvement with Pareto Optimization for Performance-Cost Trade-offs. 2026. arXiv:2610.26770. Preprint. Read at abstract depth.
4. Liu, H. et al. SoL-Pi: Recursively Scaling Auto-Research Loops for Efficient Agent Harness. 2026. arXiv:2609.20519. Preprint. Read in full.

5. Wei, Z. et al. Self-Evolving Coding Agents via Harness-Policy Optimization. 2026. arXiv:2610.22791. Preprint. Read at abstract depth.
6. Tayebati, S. et al. Self-Healing Harness for Runtime Oversight of Agent Self-Modification. 2026. arXiv:2609.24130. Preprint. Read at abstract depth.
7. Wang, X. et al. Beneath the Surface: What We Overlook When We Talk About Agent Evolution. 2026. arXiv:2611.07624. Preprint. Read at abstract depth.
8. Karten, Seth, Zhang, Alex L., Thomas, Kevin, Müller, Sebastian, Bakouch, Elie, Auras, Daniel, Senghaas, Mika, Obeid, Fares, Dunas, Konstantin, Hagemann, Johannes, and Jaghouar, Sami. Prime Agent: A Self-Improving RLM Harness. 2026. arXiv:2608.23552. Technical report. Section 3.5 read in full.
9. Dabirsiaghi, A. Compaction Laundering: Provenance Destruction in Context-Compacting Coding Agents. 2026. arXiv:2610.29869. Preprint. Read at abstract depth.
10. Zha, Y. et al. Do Agents Still Follow the Rules? Benchmarking Rule Compliance under Agent Context Compaction. 2026. arXiv:2611.03223. Preprint. Read at abstract depth.
11. The MITRE Corporation. MITRE ATT&CK for Enterprise. https://attack.mitre.org. Knowledge base of adversary tactics and techniques.
12. The MITRE Corporation. MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems. https://atlas.mitre.org. Knowledge base of adversary techniques against AI systems.
13. OWASP Foundation. OWASP Top 10 for Large Language Model Applications, 2025 edition. https://genai.owasp.org, 2025.
14. Bradner, S. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, IETF, 1997. Updated by RFC 8174.
15. Leiba, B. Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. RFC 8174, IETF, 2017.
16. Chen, H. et al. When the Critic Falls Behind: Co-Evolving Rubrics and Harnesses for Continual Agent Self-Improvement. 2026. arXiv:2610.36826. Preprint. Read at abstract depth.
17. Zhang, J. et al. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. 2025. arXiv:2505.22954. Preprint.
18. Cai, Q. et al. A Theory of Reliable Self-Evolution for Agent Harnesses. 2026. arXiv:2609.08175. Preprint. Read at abstract depth.
19. Spine authors. Spine: Compiling Subtask Trees into Structured Context for Long-Horizon Coding Agents. 2026. arXiv:2610.37224. Preprint. Taken from earlier project notes; author list not verified.
20. Belcak, P. et al. You Don't Need Long Context. 2026. arXiv:2611.05306. Preprint. Read in full.
21. Zhuang, Y. et al. AgentRewind: Recoverable Execution for Long-Horizon LLM Agents. 2026. arXiv:2608.14380. Preprint. Read in full.
22. Temporal Technologies. Temporal: Durable Execution Platform. https://temporal.io. Software.
23. Satran, J., Meth, K., Sapuntzakis, C., Chadalapaka, M., and Zeidner, E. Internet Small Computer Systems Interface (iSCSI). RFC 3720, IETF, 2004. Defines the CRC32C polynomial used for record framing.
24. Dunn, Olive Jean. Multiple Comparisons among Means. *Journal of the American Statistical Association*, 56(293):52–64, 1961.
25. Aho, Alfred V. and Corasick, Margaret J. Efficient String Matching: An Aid to Bibliographic Search. *Communications of the ACM*, 18(6):333–340, 1975.
26. Compact-Bench authors. Compact-Bench. 2026. arXiv:2610.31710. Preprint. Seen in search, not read; author list and full title not verified.